WEEK 4: SUPERMARKET MANAGEMENT SYSTEM

Logeshwari D (E25AI121)

1. Scenario

The Supermarket Management System needs a system to manage customer purchases and sales. Customers can purchase multiple products, and each sale maintains details such as sale date, purchased products, quantity, price, and total bill amount.

The system helps the supermarket manage customer purchase information, calculate the total bill amount, update payment status, and generate customer-wise and product-wise sales reports.

---

2. Design Customer, Sales and Sale_Details Tables

Customer Table

The CUSTOMER table stores information about customers who purchase products from the supermarket.

Attribute	Description
CUSTOMER_ID	Unique customer identifier
CUSTOMER_NAME	Name of the customer
PHONE	Customer contact number
EMAIL	Customer email address

```
SQL> CREATE TABLE CUSTOMER (
  2      CUSTOMER_ID VARCHAR2(10) PRIMARY KEY,
  3      CUSTOMER_NAME VARCHAR2(50) NOT NULL,
  4      PHONE VARCHAR2(15) NOT NULL,
  5      EMAIL VARCHAR2(50) UNIQUE
  6  );

Table created.

SQL> DESC CUSTOMER;
 Name                                      Null?    Type
 ----------------------------------------- -------- ----------------
 CUSTOMER_ID                               NOT NULL VARCHAR2(10)
 CUSTOMER_NAME                             NOT NULL VARCHAR2(50)
 PHONE                                     NOT NULL VARCHAR2(15)
 EMAIL                                              VARCHAR2(50)

SQL>
```

Sales Table

The SALES table stores overall information about each customer purchase.

Attribute	Description
SALE_ID	Unique sale identifier
CUSTOMER_ID	Customer reference
SALE_DATE	Date of purchase
TOTAL_AMOUNT	Total bill amount
PAYMENT_STATUS	Paid/Pending

```
SQL> CREATE TABLE SALES (
  2      SALE_ID VARCHAR2(10) PRIMARY KEY,
  3      CUSTOMER_ID VARCHAR2(10) NOT NULL,
  4      SALE_DATE DATE NOT NULL,
  5      TOTAL_AMOUNT NUMBER(10,2),
  6      PAYMENT_STATUS VARCHAR2(20),
  7      CONSTRAINT FK_SALES_CUSTOMER
  8          FOREIGN KEY (CUSTOMER_ID)
  9          REFERENCES CUSTOMER(CUSTOMER_ID),
 10      CONSTRAINT CHK_PAYMENT_STATUS
 11          CHECK (PAYMENT_STATUS IN ('Paid', 'Pending'))
 12  );

Table created.

SQL> DESC SALES;
 Name                                      Null?    Type
 ----------------------------------------- -------- ----------------
 SALE_ID                                   NOT NULL VARCHAR2(10)
 CUSTOMER_ID                               NOT NULL VARCHAR2(10)
 SALE_DATE                                 NOT NULL DATE
 TOTAL_AMOUNT                                       NUMBER(10,2)
 PAYMENT_STATUS                                     VARCHAR2(20)
```

Sale_Details Table

The SALE_DETAILS table stores individual products included in each sale.

Attribute	Description
SALE_DETAIL_ID	Unique sale detail identifier
SALE_ID	Sale reference
PRODUCT_ID	Product reference
QUANTITY	Number of products purchased
PRICE	Product price

```
SQL> CREATE TABLE SALE_DETAILS (
  2      SALE_DETAIL_ID VARCHAR2(10) PRIMARY KEY,
  3      SALE_ID VARCHAR2(10) NOT NULL,
  4      PRODUCT_ID VARCHAR2(10) NOT NULL,
  5      QUANTITY NUMBER NOT NULL,
  6      PRICE NUMBER(10,2) NOT NULL,
  7      CONSTRAINT FK_SALEDETAIL_SALE
  8          FOREIGN KEY (SALE_ID)
  9          REFERENCES SALES(SALE_ID),
 10      CONSTRAINT FK_SALEDETAIL_PRODUCT
 11          FOREIGN KEY (PRODUCT_ID)
 12          REFERENCES PRODUCT(PRODUCT_ID),
 13      CONSTRAINT CHK_SALE_QUANTITY
 14          CHECK (QUANTITY > 0),
 15      CONSTRAINT CHK_SALE_PRICE
 16          CHECK (PRICE > 0)
 17  );

Table created.

SQL> DESC SALE_DETAILS;
 Name                                      Null?    Type
 ----------------------------------------- -------- ----------------
 SALE_DETAIL_ID                            NOT NULL VARCHAR2(10)
 SALE_ID                                   NOT NULL VARCHAR2(10)
 PRODUCT_ID                                NOT NULL VARCHAR2(10)
 QUANTITY                                  NOT NULL NUMBER
 PRICE                                     NOT NULL NUMBER(10,2)
```

3. Establish Relationships

Customer and Sales

One customer can make multiple purchases.

Relationship:

CUSTOMER (1) -------- (Many) SALES

Sales and Products

One sale can contain multiple products, and one product can appear in multiple sales.

Relationship:

SALES (Many) -------- (Many) PRODUCT

This many-to-many relationship is implemented using the SALE_DETAILS table.

Product and Sale_Details

PRODUCT

|

| PRODUCT_ID

↓

SALE_DETAILS

↑

| SALE_ID

|

SALES

```
SQL> SELECT CONSTRAINT_NAME, TABLE_NAME, R_CONSTRAINT_NAME
  2  FROM USER_CONSTRAINTS
  3  WHERE TABLE_NAME IN ('CUSTOMER','SALES','SALE_DETAILS')
  4  AND CONSTRAINT_TYPE = 'R';

CONSTRAINT_NAME                TABLE_NAME
------------------------------ ------------------------------
R_CONSTRAINT_NAME
------------------------------
FK_SALES_CUSTOMER              SALES
SYS_C007217

FK_SALEDETAIL_SALE             SALE_DETAILS
SYS_C007222

FK_SALEDETAIL_PRODUCT          SALE_DETAILS
SYS_C007179
```

4. Apply Constraints

The following constraints are applied to maintain data accuracy:

Primary Key

- CUSTOMER_ID uniquely identifies each customer.
- SALE_ID uniquely identifies each sale.
- SALE_DETAIL_ID uniquely identifies each sale detail.

Foreign Key

- CUSTOMER_ID in SALES references CUSTOMER.
- SALE_ID in SALE_DETAILS references SALES.
- PRODUCT_ID in SALE_DETAILS references the existing PRODUCT table.

NOT NULL

Important fields such as:

- CUSTOMER_NAME
- PHONE
- CUSTOMER_ID
- SALE_DATE
- QUANTITY
- PRICE

should not be left empty.

CHECK

The quantity and price must be greater than zero.

QUANTITY > 0

PRICE > 0

The payment status is restricted to:

Paid

Pending

```
SQL> SELECT CONSTRAINT_NAME, TABLE_NAME, CONSTRAINT_TYPE
  2  FROM USER_CONSTRAINTS
  3  WHERE TABLE_NAME IN ('CUSTOMER', 'SALES', 'SALE_DETAILS');

CONSTRAINT_NAME                TABLE_NAME                     C
------------------------------ ------------------------------ -
SYS_C007215                    CUSTOMER                       C
SYS_C007216                    CUSTOMER                       C
SYS_C007217                    CUSTOMER                       P
SYS_C007218                    CUSTOMER                       U
SYS_C007219                    SALES                          C
SYS_C007220                    SALES                          C
CHK_PAYMENT_STATUS             SALES                          C
SYS_C007222                    SALES                          P
FK_SALES_CUSTOMER              SALES                          R
SYS_C007224                    SALE_DETAILS                   C
SYS_C007225                    SALE_DETAILS                   C

CONSTRAINT_NAME                TABLE_NAME                     C
------------------------------ ------------------------------ -
SYS_C007226                    SALE_DETAILS                   C
SYS_C007227                    SALE_DETAILS                   C
CHK_SALE_QUANTITY              SALE_DETAILS                   C
CHK_SALE_PRICE                 SALE_DETAILS                   C
SYS_C007230                    SALE_DETAILS                   P
FK_SALEDETAIL_SALE             SALE_DETAILS                   R
FK_SALEDETAIL_PRODUCT          SALE_DETAILS                   R

18 rows selected.
```

5. Inserting Values

Sample customer records are inserted into the CUSTOMER table.

```
SQL> SELECT * FROM CUSTOMER;

CUSTOMER_I CUSTOMER_NAME                                           PHONE
---------- ---------------------------------------------------- ----------
EMAIL
--------------------------------------------------
C001       Arun                                                 9876543210
arun@gmail.com

C002       Priya                                                9876543211
priya@gmail.com

C003       Karthik                                              9876543212
karthik@gmail.com


CUSTOMER_I CUSTOMER_NAME                                           PHONE
---------- ---------------------------------------------------- ----------
EMAIL
--------------------------------------------------
C004       Divya                                                9876543213
divya@gmail.com

C005       Rahul                                                9876543214
rahul@gmail.com
```

Sample sales records are inserted into the SALES table.

```
SQL> SELECT * FROM SALES;

SALE_ID    CUSTOMER_I SALE_DATE TOTAL_AMOUNT PAYMENT_STATUS
---------- ---------- --------- ------------ ----------------
S001       C001       01-SEP-26          415 Paid
S002       C002       02-SEP-26          300 Paid
S003       C003       03-SEP-26          340 Pending
S004       C004       04-SEP-26          470 Paid
S005       C005       05-SEP-26          360 Paid

SQL>
```

Sample purchased product records are inserted into the SALE_DETAILS table.

```
SQL> SELECT * FROM SALE_DETAILS;

SALE_DETAI SALE_ID    PRODUCT_ID   QUANTITY      PRICE
---------- ---------- ---------- ---------- ----------
SD001      S001       P001                1        350
SD002      S001       P002                1         65
SD003      S002       P004                1        120
SD004      S002       P005                1        180
SD005      S003       P003                2         55
SD006      S003       P007                1         40
SD007      S003       P008                1         30
SD008      S004       P010                1        220
SD009      S004       P009                1         80
SD010      S004       P005                1        180
SD011      S005       P015                1        280

SALE_DETAI SALE_ID    PRODUCT_ID   QUANTITY      PRICE
---------- ---------- ---------- ---------- ----------
SD012      S005       P006                1         25
SD013      S005       P008                1         30

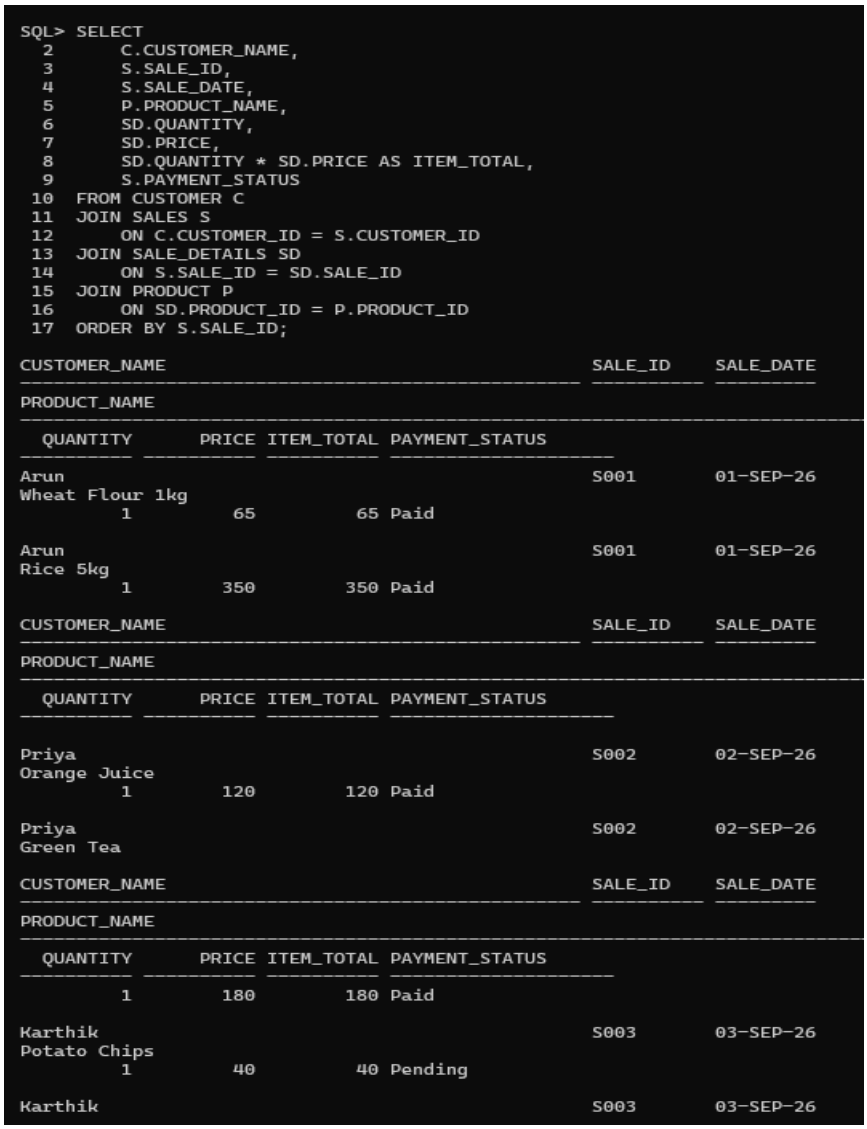
13 rows selected.
```

After inserting the records, the contents of the tables are displayed to verify that the data has been added successfully.

6. Manage Customer Sales

The following operations are performed:

1. Create a new sale for a customer.
2. Add multiple products to a sale.
3. Calculate the total bill amount.
4. Update payment status.
5. Update product quantity.
6. View complete sale details.



```
SQL> SELECT
  2      C.CUSTOMER_NAME,
  3      S.SALE_ID,
  4      S.SALE_DATE,
  5      P.PRODUCT_NAME,
  6      SD.QUANTITY,
  7      SD.PRICE,
  8      SD.QUANTITY * SD.PRICE AS ITEM_TOTAL,
  9      S.PAYMENT_STATUS
 10  FROM CUSTOMER C
 11  JOIN SALES S
 12      ON C.CUSTOMER_ID = S.CUSTOMER_ID
 13  JOIN SALE_DETAILS SD
 14      ON S.SALE_ID = SD.SALE_ID
 15  JOIN PRODUCT P
 16      ON SD.PRODUCT_ID = P.PRODUCT_ID
 17  ORDER BY S.SALE_ID;

CUSTOMER_NAME                                      SALE_ID    SALE_DATE
-------------------------------------------------- ---------- ---------
PRODUCT_NAME
--------------------------------------------------------------------------------
  QUANTITY      PRICE ITEM_TOTAL PAYMENT_STATUS
---------- ---------- ---------- --------------------
Arun                                               S001       01-SEP-26
Wheat Flour 1kg
         1         65         65 Paid

Arun                                               S001       01-SEP-26
Rice 5kg
         1        350        350 Paid

CUSTOMER_NAME                                      SALE_ID    SALE_DATE
-------------------------------------------------- ---------- ---------
PRODUCT_NAME
--------------------------------------------------------------------------------
  QUANTITY      PRICE ITEM_TOTAL PAYMENT_STATUS
---------- ---------- ---------- --------------------

Priya                                              S002       02-SEP-26
Orange Juice
         1        120        120 Paid

Priya                                              S002       02-SEP-26
Green Tea

CUSTOMER_NAME                                      SALE_ID    SALE_DATE
-------------------------------------------------- ---------- ---------
PRODUCT_NAME
--------------------------------------------------------------------------------
  QUANTITY      PRICE ITEM_TOTAL PAYMENT_STATUS
---------- ---------- ---------- --------------------
         1        180        180 Paid

Karthik                                            S003       03-SEP-26
Potato Chips
         1         40         40 Pending

Karthik                                            S003       03-SEP-26
```

```
CUSTOMER_NAME                                      SALE_ID    SALE_DATE
-------------------------------------------------- ---------- ---------
PRODUCT_NAME
--------------------------------------------------------------------------------
  QUANTITY      PRICE ITEM_TOTAL PAYMENT_STATUS
---------- ---------- ---------- --------------------
Divya                                              S004       04-SEP-26
Chocolate
         1         80         80 Paid

Divya                                              S004       04-SEP-26
Shampoo
         1        220        220 Paid

CUSTOMER_NAME                                      SALE_ID    SALE_DATE
-------------------------------------------------- ---------- ---------
PRODUCT_NAME
--------------------------------------------------------------------------------
  QUANTITY      PRICE ITEM_TOTAL PAYMENT_STATUS
---------- ---------- ---------- --------------------

Divya                                              S004       04-SEP-26
Green Tea
         1        180        180 Paid

Rahul                                              S005       05-SEP-26
Biscuits

CUSTOMER_NAME                                      SALE_ID    SALE_DATE
-------------------------------------------------- ---------- ---------
PRODUCT_NAME
--------------------------------------------------------------------------------
  QUANTITY      PRICE ITEM_TOTAL PAYMENT_STATUS
---------- ---------- ---------- --------------------
         1         30         30 Paid

Rahul                                              S005       05-SEP-26
Laundry Detergent
         1        280        280 Paid

Rahul                                              S005       05-SEP-26

CUSTOMER_NAME                                      SALE_ID    SALE_DATE
-------------------------------------------------- ---------- ---------
PRODUCT_NAME
--------------------------------------------------------------------------------
  QUANTITY      PRICE ITEM_TOTAL PAYMENT_STATUS
---------- ---------- ---------- --------------------
Mineral Water
         1         25         25 Paid


13 rows selected.

SQL> |
```

7. Calculate Total Bill Amount

The total bill amount is calculated using:

TOTAL AMOUNT = QUANTITY × PRICE For multiple products, the individual product amounts are added to calculate the complete sale amount.

```
SQL> SELECT
  2      S.SALE_ID,
  3      C.CUSTOMER_NAME,
  4      SUM(SD.QUANTITY * SD.PRICE) AS CALCULATED_TOTAL
  5  FROM SALES S
  6  JOIN CUSTOMER C
  7      ON S.CUSTOMER_ID = C.CUSTOMER_ID
  8  JOIN SALE_DETAILS SD
  9      ON S.SALE_ID = SD.SALE_ID
 10  GROUP BY S.SALE_ID, C.CUSTOMER_NAME
 11  ORDER BY S.SALE_ID;

SALE_ID    CUSTOMER_NAME                                      CALCULATED_TOTAL
---------- -------------------------------------------------- ----------------
S001       Arun                                                            415
S002       Priya                                                           300
S003       Karthik                                                         180
S004       Divya                                                           480
S005       Rahul                                                           335

SQL> |
```

8. Update Operation

The UPDATE command is used to modify sale information.

Example:

Change payment status from **Pending** to **Paid**.

```
SQL> SELECT SALE_ID, CUSTOMER_ID, PAYMENT_STATUS
  2  FROM SALES
  3  WHERE SALE_ID = 'S003';

SALE_ID    CUSTOMER_I PAYMENT_STATUS
---------- ---------- --------------------
S003       C003       Paid

SQL> UPDATE SALES
  2  SET PAYMENT_STATUS = 'Paid'
  3  WHERE SALE_ID = 'S003';

1 row updated.

SQL>
SQL> COMMIT;

Commit complete.

SQL> SELECT SALE_ID, CUSTOMER_ID, PAYMENT_STATUS
  2  FROM SALES
  3  WHERE SALE_ID = 'S003';

SALE_ID    CUSTOMER_I PAYMENT_STATUS
---------- ---------- --------------------
S003       C003       Paid
```

9. Delete Operation

The DELETE command is used to remove cancelled sales.

Example:

Delete a cancelled sale from the system.

The corresponding records in SALE_DETAILS are removed before deleting the sale record.

```
SQL> INSERT INTO SALES
  2  VALUES ('S006', 'C005', DATE '2026-09-06', 80, 'Pending');

1 row created.

SQL>
SQL> COMMIT;

Commit complete.

SQL> SELECT * FROM SALES
  2  WHERE SALE_ID = 'S006';

SALE_ID    CUSTOMER_I SALE_DATE TOTAL_AMOUNT PAYMENT_STATUS
---------- ---------- --------- ------------ --------------------
S006       C005       06-SEP-26           80 Pending

SQL> DELETE FROM SALES
  2  WHERE SALE_ID = 'S006';

1 row deleted.

SQL>
SQL> COMMIT;

Commit complete.

SQL> SELECT * FROM SALES
  2  WHERE SALE_ID = 'S006';

no rows selected

SQL> |
```

10. Customer Purchase History Report

This report displays the purchase history of customers.

Information displayed:

- Customer Name
- Sale ID
- Sale Date
- Total Amount
- Payment Status

The JOIN operation is used to combine the CUSTOMER and SALES tables.

```
SQL> SELECT
  2      C.CUSTOMER_NAME,
  3      S.SALE_ID,
  4      S.SALE_DATE,
  5      S.TOTAL_AMOUNT,
  6      S.PAYMENT_STATUS
  7  FROM CUSTOMER C
  8  JOIN SALES S
  9      ON C.CUSTOMER_ID = S.CUSTOMER_ID
 10  ORDER BY C.CUSTOMER_NAME, S.SALE_DATE;

CUSTOMER_NAME                                      SALE_ID    SALE_DATE
-------------------------------------------------- ---------- ---------
TOTAL_AMOUNT PAYMENT_STATUS
------------ --------------------
Arun                                               S001       01-SEP-26
         415 Paid

Divya                                              S004       04-SEP-26
         470 Paid

Karthik                                            S003       03-SEP-26
         340 Paid


CUSTOMER_NAME                                      SALE_ID    SALE_DATE
-------------------------------------------------- ---------- ---------
TOTAL_AMOUNT PAYMENT_STATUS
------------ --------------------
Priya                                              S002       02-SEP-26
         300 Paid

Rahul                                              S005       05-SEP-26
         360 Paid
```

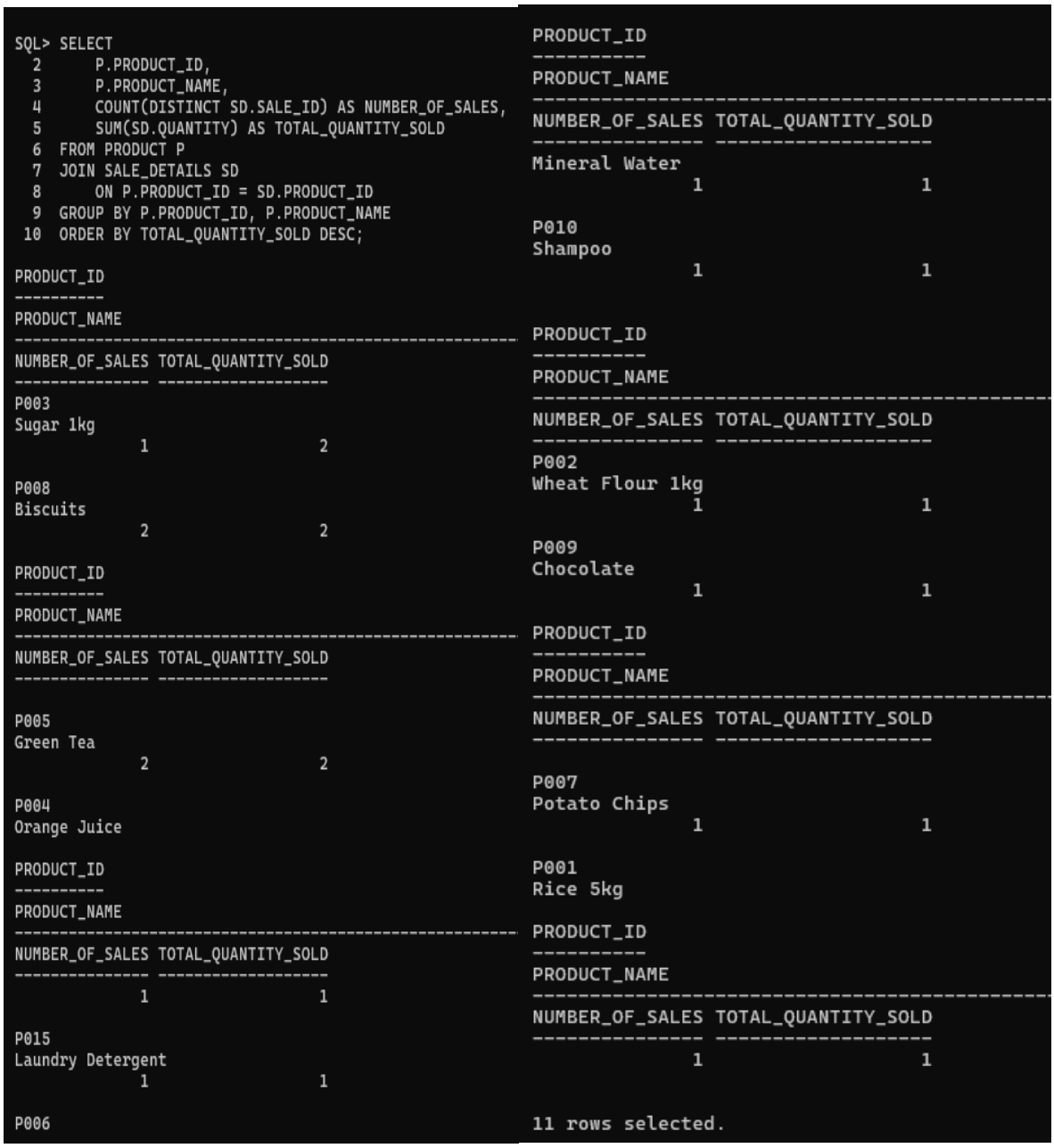
11. Product-wise Sales Report

This report provides information about the sales of each product.

Find:

- Number of times each product was sold
- Total quantity sold
- Total sales amount

The JOIN, COUNT, and SUM functions are used to generate the report.



```
SQL> SELECT
  2      P.PRODUCT_ID,
  3      P.PRODUCT_NAME,
  4      COUNT(DISTINCT SD.SALE_ID) AS NUMBER_OF_SALES,
  5      SUM(SD.QUANTITY) AS TOTAL_QUANTITY_SOLD
  6  FROM PRODUCT P
  7  JOIN SALE_DETAILS SD
  8      ON P.PRODUCT_ID = SD.PRODUCT_ID
  9  GROUP BY P.PRODUCT_ID, P.PRODUCT_NAME
 10  ORDER BY TOTAL_QUANTITY_SOLD DESC;

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------
NUMBER_OF_SALES TOTAL_QUANTITY_SOLD
--------------- -------------------
P003
Sugar 1kg
              1                   2

P008
Biscuits
              2                   2

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------
NUMBER_OF_SALES TOTAL_QUANTITY_SOLD
--------------- -------------------

P005
Green Tea
              2                   2

P004
Orange Juice

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------
NUMBER_OF_SALES TOTAL_QUANTITY_SOLD
--------------- -------------------
              1                   1

P015
Laundry Detergent
              1                   1

P006
Mineral Water
              1                   1

P010
Shampoo
              1                   1


PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------
NUMBER_OF_SALES TOTAL_QUANTITY_SOLD
--------------- -------------------
P002
Wheat Flour 1kg
              1                   1

P009
Chocolate
              1                   1

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------
NUMBER_OF_SALES TOTAL_QUANTITY_SOLD
--------------- -------------------

P007
Potato Chips
              1                   1

P001
Rice 5kg

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------
NUMBER_OF_SALES TOTAL_QUANTITY_SOLD
--------------- -------------------
              1                   1


11 rows selected.
```

13. Highest-Selling Product

This report identifies the product with the highest quantity soldThe SUM function is used to calculate the total quantity sold for each product, and the product with the highest value is identified.

```
SQL> SELECT PRODUCT_ID, PRODUCT_NAME, TOTAL_QUANTITY_SOLD
  2  FROM (
  3      SELECT
  4          P.PRODUCT_ID,
  5          P.PRODUCT_NAME,
  6          SUM(SD.QUANTITY) AS TOTAL_QUANTITY_SOLD
  7      FROM PRODUCT P
  8      JOIN SALE_DETAILS SD
  9          ON P.PRODUCT_ID = SD.PRODUCT_ID
 10      GROUP BY P.PRODUCT_ID, P.PRODUCT_NAME
 11      ORDER BY TOTAL_QUANTITY_SOLD DESC
 12  )
 13  WHERE ROWNUM = 1;

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------------------------
TOTAL_QUANTITY_SOLD
-------------------
P005
Green Tea
                  2
```

12. Customer Purchase Analysis

This report analyzes customer purchasing information.

Find:

- Number of purchases made by each customer
- Total amount spent by each customer
- Highest-spending customer

The COUNT and SUM functions are used to analyze customer purchases

```
SQL> SELECT
  2      C.CUSTOMER_ID,
  3      C.CUSTOMER_NAME,
  4      COUNT(S.SALE_ID) AS TOTAL_SALES,
  5      SUM(S.TOTAL_AMOUNT) AS TOTAL_PURCHASE_AMOUNT
  6  FROM CUSTOMER C
  7  JOIN SALES S
  8      ON C.CUSTOMER_ID = S.CUSTOMER_ID
  9  GROUP BY C.CUSTOMER_ID, C.CUSTOMER_NAME
 10  ORDER BY TOTAL_PURCHASE_AMOUNT DESC;

CUSTOMER_I CUSTOMER_NAME                                      TOTAL_SALES
---------- -------------------------------------------------- -----------
TOTAL_PURCHASE_AMOUNT
---------------------
C004       Divya                                                        1
                  470

C001       Arun                                                         1
                  415

C005       Rahul                                                        1
                  360


CUSTOMER_I CUSTOMER_NAME                                      TOTAL_SALES
---------- -------------------------------------------------- -----------
TOTAL_PURCHASE_AMOUNT
---------------------
C003       Karthik                                                      1
                  340

C002       Priya                                                        1
                  300
```

Conclusion

The Week 4 Supermarket Management System extends the previous product, category, supplier and inventory management modules by adding customer sales management. The CUSTOMER, SALES and SALE_DETAILS tables were created with appropriate primary key, foreign key, NOT NULL, UNIQUE and CHECK constraints.

The system supports recording sales, maintaining sale details, calculating bill amounts, updating payment status and deleting sale records. Various reports were also generated, including customer purchase history, product-wise sales, customer purchase analysis and the highest-selling product.

Thus, the system provides an organized way to manage supermarket sales and analyze customer and product purchasing information.